HelixQA proxy test bank — helix_proxy

Revision: 1 **Last modified:** 2026-07-01T09:00:23Z

Anti-bluff HelixQA (§11.4.27) coverage for the helix_proxy data plane: Squid HTTP/HTTPS forward+cache proxy (`:53128`) and Dante SOCKS5 proxy (`:51080`). This directory documents the bank, the runner, and the current honest §11.4.3 SKIP blocking a real HelixQA run.

1. Investigation facts (submodules/helix_qa)

- **Bank format:** HelixQA YAML test bank. Required top-level keys `version`, `name`, `test_cases`; per-step executable action `"http: <METHOD> <PATH>"` (`ActionTypeHTTP`). Source of truth: `submodules/helix_qa/pkg/testbank/schema.go` (`ActionTypeHTTP` = line 224, `ParseAction` = line 376; assertion fields `expect_status` / `expect_body_contains` / `expect_json_path` = lines 309–321).
- **Run command (LLM-free, no browser/device):** `helixqa http --bank <bank.yaml> --base-url <URL> [--json --verbose]` (`submodules/helix_qa/cmd/helixqa/http.go`). It loads a bank, runs every `http:` step through `pkg/autonomous.HTTPExecutor` against `--base-url`, and reports per-case PASS/FAIL with the real status code. (`helixqa run` needs Playwright/ADB; `helixqa autonomous` needs an LLM — neither fits a proxy.)
- **Can it target a proxy?** Yes, via env — the load-bearing mechanism: `HTTPExecutor` builds `&http.Client{Timeout:30s}` with a **nil Transport** (`pkg/autonomous/http_executor.go:110`), so Go uses `http.DefaultTransport` whose `Proxy: http.ProxyFromEnvironment` reads `HTTP_PROXY` / `HTTPS_PROXY`. Setting those to the live proxy forwards every request THROUGH it to the `--base-url` upstream; the upstream's real status code is the proof of forwarding. **Empirically proven this session** with a stdlib replica of that exact client: HTTP fwd → 204, HTTPS CONNECT → 200, SOCKS5 → 204 (`qa-results/helixqa/20260701T085355Z/mech_*.txt`).
- **Prerequisite / blocker (see §4):** the `helixqa` binary does not build in this checkout.

2. Deliverables

Path	Purpose
`tools/helixqa/banks/proxy.yaml`	Canonical bank — all 4 features (HTTP forward, HTTPS-through, SOCKS5, cache), 6 cases, `helixqa list`/inventory.
`tools/helixqa/banks/routes/*.yaml`	Per-transport execution slices (one `--base-url` each) the runner feeds to `helixqa http`.
`tools/helixqa/runner/run_proxy_bank.sh`	Builds `helixqa` (if siblings resolve) and runs each route bank through the live proxy; else honest SKIP.

Route → target → transport:

Route bank	`--base-url`	proxy env
`proxy_http_forward.yaml`	`http://connectivitycheck.gstatic.com`	`HTTP_PROXY=http://127.0.`
`proxy_https_through.yaml`	`https://connectivitycheck.gstatic.com`	`HTTPS_PROXY=http://127.0`
`proxy_socks5.yaml`	`http://connectivitycheck.gstatic.com`	`HTTP_PROXY=socks5://127.`
`proxy_socks5_https.yaml`	`https://connectivitycheck.gstatic.com`	`HTTPS_PROXY=socks5://127`
`proxy_cache.yaml`	`http://www.gnu.org`	`HTTP_PROXY=http://127.0.`

Targets and expected codes are the project's own sanctioned values (`tests/verify-proxy.sh`, `tests/comprehensive-test.sh`) and were re-confirmed live this session.

3. Cache verification is sink-side (§11.4.69)

The `HTTPExecutor` asserts only status/body/json-path — NOT response headers or Squid's access.log — so a cache HIT cannot be asserted by an `http:` step. The two cache `http:` steps prove the object is served consistently THROUGH the proxy; the DECISIVE HIT proof is a Squid `TCP_*HIT` for the exact URL in `proxy-squid:/var/log/squid/access.log` (plus `./cachectl stats`), captured sink-side by the runner exactly as `tests/comprehensive-test.sh` does. HTTPS bodies are CONNECT-tunnelled and never cacheable by Squid, so the cache target is deliberately plain HTTP.

4. Current status — honest SKIP (§11.4.3), blocker + unblock

Running an actual HelixQA session against the live proxy is **BLOCKED**: the `helixqa` CLI cannot be built here. `submodules/helix_qa/go.mod` `replace`s six own-org sibling modules to `../<name>` paths that are **not vendored** in helix_proxy, and both `pkg/testbank` (`manager.go:11`) and `pkg/autonomous` (`coordinator.go:15`) import `digital.vasic.docprocessor`, so even a minimal build fails (captured `go build` output in the run's `build*.log`).

Missing siblings (present: `challenges`, `containers`): `doc_processor`, `llm_orchestrator`, `llm_provider`, `llms_verifier`, `vision_engine`, `security`.

Unblock (operator/conductor): vendor those six own-org modules as siblings under `submodules/` (SSH, per §11.4.28(C)/§11.4.36), then re-run `tools/helixqa/runner/run_proxy_bank.sh` — it builds `helixqa` and executes the bank against the live proxy automatically, writing `result.json` + evidence under `qa-results/helixqa/<run-ts>/`.

Not a bluff: the proxy data plane itself is proven working this session (HTTP 204 / HTTPS 200 / SOCKS5 204 through the executor's exact client, plus curl 204/200/204 and cache target 200/33 766 B). The SKIP is about the HelixQA *harness build*, not the proxy.

Sources verified 2026-07-01

- `submodules/helix_qa/cmd/helixqa/http.go`, `pkg/autonomous/http_executor.go`, `pkg/testbank/schema.go`, `README.md`, `CLAUDE.md` (bank format + run command).
- `tests/verify-proxy.sh`, `tests/comprehensive-test.sh` (sanctioned targets + cache-HIT convention).
- Live proxy probes + executor-client mechanism proof (`qa-results/helixqa/20260701T085355Z/mech_*.txt`).